Witch Hunt (คืนล่าแม่มด)

กลางดึกคืนหนึ่ง ณ ห้อง TA มีพี่ ๆ และน้อง ๆ ในค่ายนั่งล้อมวงกันอยู่หลายคน พวกเขาเล่นกิจกรรมสนุก ๆ กัน เพื่อคลายเครียดหลังจากนั่งทำโจทย์มาเป็นเวลานาน ซึ่งคือการเล่น**บอร์ดเกมมนุษย์หมาป่า (Werewolf)** นั่นเอง ในเกมนี้มันก็จะมีบทบาทน่าเล่นมากมายเช่น กามเทพ, บอดี้การ์ด, เซียร์ แต่มีอยู่บทหนึ่งที่แทบไม่มีประโยชน์เลย นั่นคือ**บทแม่มด** เพราะแม่มดมีขวดยาเพียงสองขวด คือยาชุบชีวิตและยาพิษ และแต่ละขวด**ใช้ได้เพียงครั้งเดียวตลอดทั้งเกม แม้แต่ moderator ยังไม่ชอบเลย (ฉันเอง) เพราะทุกคนชอบไม่ใช้จนจบเกม** แต่ในคืนนั้น มีน้องคนนึงที่ได้บทแม่มด อยู่ ๆ เลือกใช้ยาพิษ และดันไปเลือก**บอมเมอร์**เข้าพอดี บอมเมอร์จึงระเบิดพาคนที่นั่งข้าง ๆ ตายตามไปด้วยอีก 2 คน (ตามบทบาทของบอมเมอร์) ในคืนเดียวกันนั้นเอง หมาป่าก็ดันไปเลือกกัด**คนที่มีคู่รักจากกามเทพ** คู่รักของเขาจึงตรอมใจตายตามไปอีกคน ส่วนบอดี้การ์ดก็ปกป้องใครไว้ไม่ได้เลยสักคน สุดท้ายเกมนั้น**ตายไปทั้งหมด 5 คนในคืนเดียว** และหมาป่าก็ชนะไปอย่างง่ายดาย สรุปว่า **“แม่มดฆ่าคนได้เยอะกว่าหมาป่าซะอีก”**

ดังนั้นเขาเลยเปลี่ยนใจจาก**การล่าหมาป่า**มาเป็น**การล่าแม่มด (Witch Hunt)** แทน เพื่อความเป็นระเบียบ ผู้เล่นทั้งหมด N คนจะนั่งล้อมวงกันเป็นวงกลม และ**กำหนดหมายเลขที่นั่งให้ผู้เล่นแต่ละคนตั้งแต่ 1 ถึง N** ตามเข็มนาฬิกา แต่ก็ไม่มีใครรู้เลยว่าใครกันแน่ที่เป็นแม่มด



รายละเอียดของเหตุการณ์

Mode 1: When The Witch Hunt Starts

เนื่องจากว่าทุกคนตัดสินใจไม่ได้สักทีว่าจะโหวตใครออกในแต่ละคืน ทุกคนเลยตกลงกันว่า จะใช้**การนับ**ในการตัดสินว่าใครจะถูก**ไล่ออกจากวง**แทน แต่ทว่า ในทุก ๆ คืน จะไล่ผู้เล่นออกจากวงได้เพียง**คืนละ 1 คน**เท่านั้น เพราะหากไล่ออกทีละหลายคน เกมก็จะจบเร็วเกินไปจนไม่สนุก ผู้เล่นที่ถูกไล่ออกจากวงไปแล้วจะต้องไปนั่งรอดูอยู่นอกวง และ**ไม่สามารถกลับเข้ามาในวงได้อีก**

โดยผู้ดำเนินเกมมีวิธีการคือจะ**เริ่มนับ 1 จากผู้เล่นที่นั่งหมายเลข 1 (ในคืนแรก) และนับวนไปตามเข็มนาฬิกา ทุกครั้งที่นับครบ M คน โดยไม่นับที่นั่งที่ว่างไปแล้ว** ผู้เล่นคนนั้นจะถูกไล่ออกจากวงทันที เมื่อไล่ออกเสร็จในแต่ละคืน **ผู้ดำเนินเกมจะจำตำแหน่งนั้นไว้ และในคืนถัดไปจะเริ่มนับ 1 ใหม่จากตำแหน่งถัดไปของผู้เล่นที่เพิ่งถูกไล่ออกล่าสุด และเป็นตำแหน่งที่ยังมีผู้เล่นนั่งอยู่** ซึ่งจะทำเช่นนี้ไปเรื่อย ๆ ทุกคืน จนกระทั่ง**เหลือผู้เล่นในวงเพียงคนเดียว** จึงต้องหาว่า **ผู้เล่นที่นั่งหมายเลขใดจะเป็นคนสุดท้ายที่เหลือรอด เพราะคนสุดท้ายที่เหลือรอดคือแม่มดนั่นเอง**

Mode 2: Witch Redemption

เนื่องจากว่าแม่มดยังไม่อยากถูกพบตัว เขาจะสามารถทำการ redemption โดยในบางคืน**แม่มดจะเรียกผู้เล่นที่ถูกไล่ออกไปแล้วให้กลับเข้ามาในวง** เพื่อใช้เป็นตัวล่อ ตัวเองจะได้ไม่ต้องเป็นคนสุดท้ายที่เหลืออยู่ ผู้เล่นที่ถูกเรียกกลับจะเดินกลับมานั่งได้ภายในคืนนั้น **ก่อนจะเริ่มนับ และไล่ผู้เล่นออกในคืนถัดไป (จะเริ่มนับใหม่หลังจากที่ทำการ redemption ครบแล้วจึงจะเริ่มนับ 1 ใหม่จากตำแหน่งถัดไปของผู้เล่นที่เพิ่งถูกไล่ออกล่าสุด** **ดูได้ที่คำอธิบายตัวอย่างข้อที่ 3****)**

โดยมีเงื่อนไขดังนี้:

1. แม่มดจะตอบโต้เมื่อถึง **คืนที่** p_i (กำหนดให้แม่มด redemption ได้ทั้งหมด k ครั้ง)
2. ในคืนที่กำหนด**หลังจากเกิดการไล่ออกแล้ว** แม่มดจะเรียก**ผู้เล่นที่ถูกไล่ออกไปนานที่สุด** (ผู้เล่นคนแรกสุดที่ถูกไล่ออกไปก่อนหน้า และยังไม่ถูกเรียกกลับเข้ามา) โดยผู้เล่นคนนั้นจะกลับมานั่งที่ **ที่นั่งหมายเลขเดิม** ของตนเอง

3. **ข้อจำกัด:**

   - คืนเดียวกัน แม่มดเรียกคนกลับมา**ได้หลายครั้ง** (ถ้ามีตัวเลข p_i ที่ซ้ำกัน)
   - หากในวงมีผู้เล่นนั่งครบ N คนอยู่แล้ว แม่มดจะไม่สามารถเรียกใครกลับเข้ามาได้
   - หากคืนใดผู้ดำเนินเกมพบว่าเหลือผู้เล่นในวงเพียงแค่ 1 คนอยู่แล้ว**ก่อน**จะเกิดการไล่คนออก เกมจะจบลงทันที โดยการ redemption จะไม่เกิดขึ้นตั้งแต่คืนนั้นอีก

สุดท้ายแล้ว ด้วยความสามัคคีของผู้เล่นทุกคนในวง ไม่ว่าแม่มดจะเรียกตัวล่อกลับมากี่คน เกมก็จะดำเนินไปจนเหลือผู้เล่นเพียงคนเดียวอยู่ดี ดังนั้น คุณจึงต้องหาว่า **แม่มดนั่งอยู่ที่ที่นั่งหมายเลขใด**

รายละเอียดการเขียนโปรแกรม

เกรดเดอร์จะสร้างวงของผู้เล่นเป็น **Circular Singly Linked List** ที่มีหมายเลขที่นั่ง 1 ถึง N เรียงตามเข็มนาฬิกาไว้ให้ โดยแต่ละโหนดมีโครงสร้างดังนี้

```
struct Node {
        int val;  // ผู้เล่น (ที่นั่ง 1 ถึง N)
        Node* next;
};
```

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

Node* mode1(Node* head, int m)

- head: ตัวชี้ไปยังผู้เล่นที่นั่งหมายเลข 1 ซึ่งเป็นจุดเริ่มต้นของวงกลม
- m: ระยะการนับ (นับครบ m คนจะไล่ผู้เล่นคนนั้นออกจากวง)
- ฟังก์ชันนี้จะต้องคืนค่าตัวชี้ (Node*) ไปยังผู้เล่นคนสุดท้ายที่เหลืออยู่ในวง
- ฟังก์ชันนี้จะถูกเรียกเพียงครั้งเดียว เมื่อ mode = 1

Node* mode2(Node* head, int m, int k, const vector<int>& p)

- head: ตัวชี้ไปยังผู้เล่นที่นั่งหมายเลข 1
- m: ระยะการนับ
- k: จำนวนครั้งที่แม่มดใช้ redemption เรียกผู้เล่นกลับเข้าวง
- p: เวกเตอร์ขนาด k เก็บคืนที่แม่มดจะใช้ redemption เรียงจากน้อยไปมาก คือ p[0], p[1], ..., p[k-1] (เช่น p[i] = 3 หมายความว่าหลังจากไล่ผู้เล่นออกในคืนที่ 3 เสร็จ แม่มดจะเรียกผู้เล่นที่ถูกไล่ออกไปนานที่สุดกลับเข้าวงทันที)
- ฟังก์ชันนี้จะต้องคืนค่าตัวชี้ (Node*) ไปยังผู้เล่นคนสุดท้ายที่เหลืออยู่ในวง (แม่มด)
- ฟังก์ชันนี้จะถูกเรียกเพียงครั้งเดียว เมื่อ mode = 2

คุณจะสามารถเรียกใช้ฟังก์ชันต่อไปนี้ได้

Node* createNode(int v)

- v: หมายเลขที่นั่งของผู้เล่นที่ต้องการสร้าง
- ฟังก์ชันนี้จะคืนค่าตัวชี้ไปยังโหนดใหม่ที่มี val = v และ next = nullptr
- ใช้สร้างโหนดของผู้เล่นที่ถูกเรียกกลับใน mode2 แล้วนำไปต่อเข้าวงเอง

ขอบเขต

- $1 \le N \le 100$
- $1 \le M \le N$
- $1 \le k \le 100$
- $1 \le p[i] \le 100$ และ $p[i] \le p[i+1]$

ปัญหาย่อย

ปัญหาย่อย	คะแนน	ข้อกำหนดเพิ่มเติม
1	80	มีแค่ mode = 1
2	20	ไม่มีข้อกำหนดเพิ่มเติม

เกรดเดอร์ตัวอย่าง

เกรดเดอร์ตัวอย่างจะอ่านข้อมูลดังต่อไปนี้:

```
mode
N M
k
p[0]
p[1]
...
p[k-1]
```

mode คือโหมดของเกม (1 หรือ 2) ส่วน k และ p[0] ถึง p[k-1] จะมีเฉพาะเมื่อ mode = 2 โดย p แต่ละตัวอยู่คนละบรรทัด

เกรดเดอร์ตัวอย่างจะแสดงผลดังต่อไปนี้:

```
val
```

val คือหมายเลขที่นั่งของโหนดที่ฟังก์ชัน mode1 หรือ mode2 คืนค่ากลับมา

ตัวอย่างข้อมูลที่เกรดเดอร์อ่านและผลลัพธ์ที่แสดง

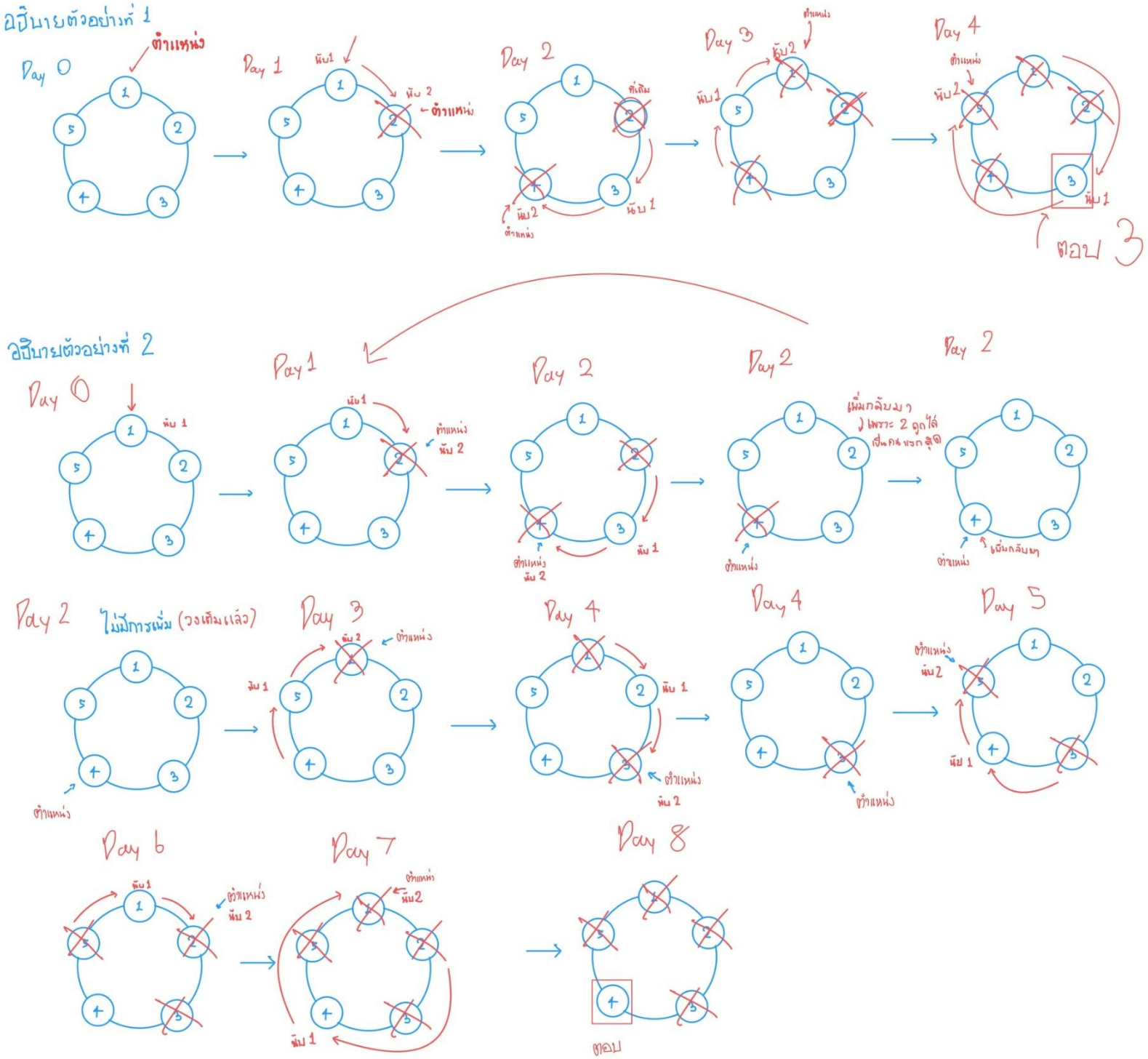
ข้อมูลนำเข้า	ข้อมูลส่งออก
1 5 2	3
2 5 2 4 2 2	4

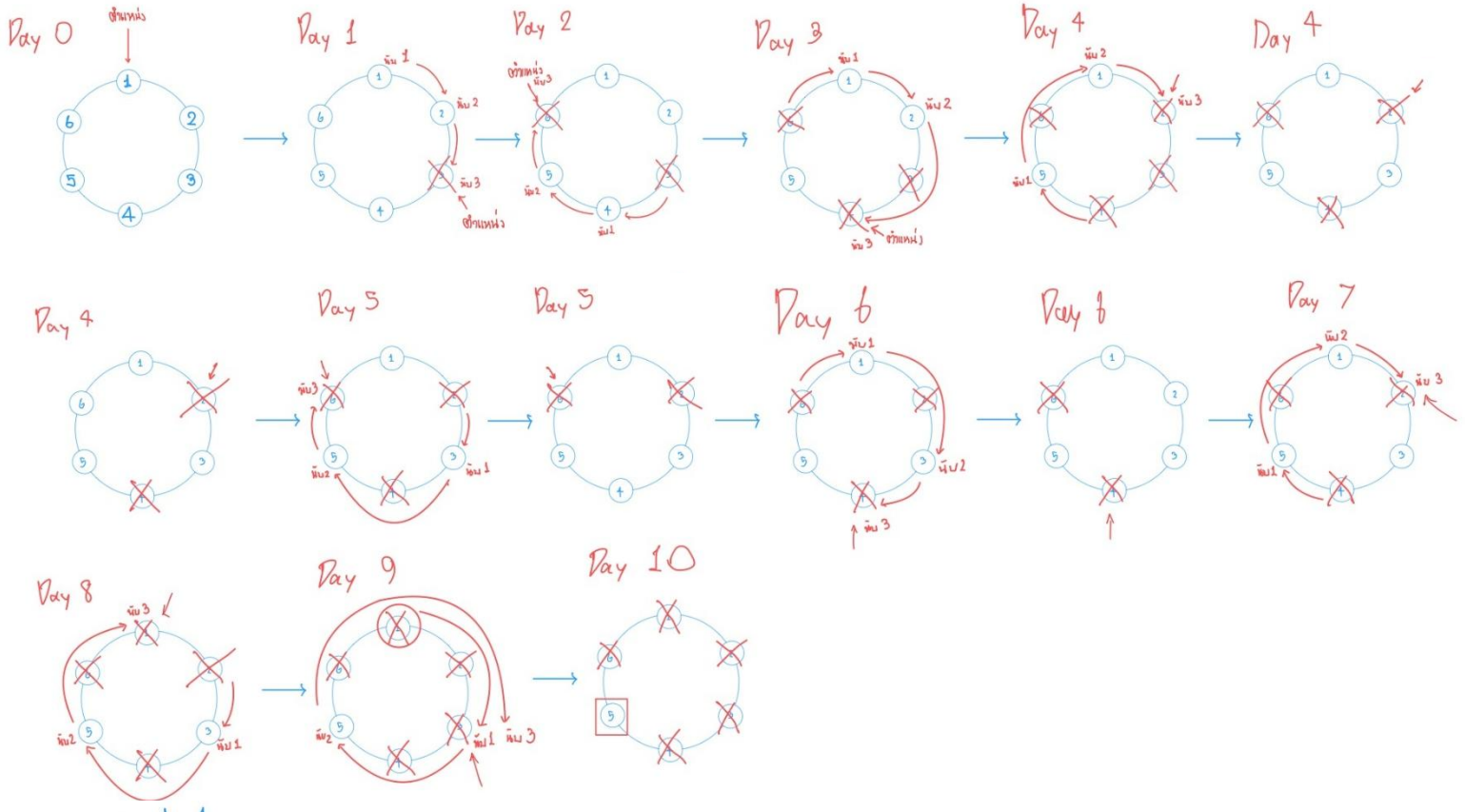
2 4	
2 6 3 4 4 4 5 6	5
2 6 3 7 4 4 5 6 9 11 100	6

ข้อกำหนด

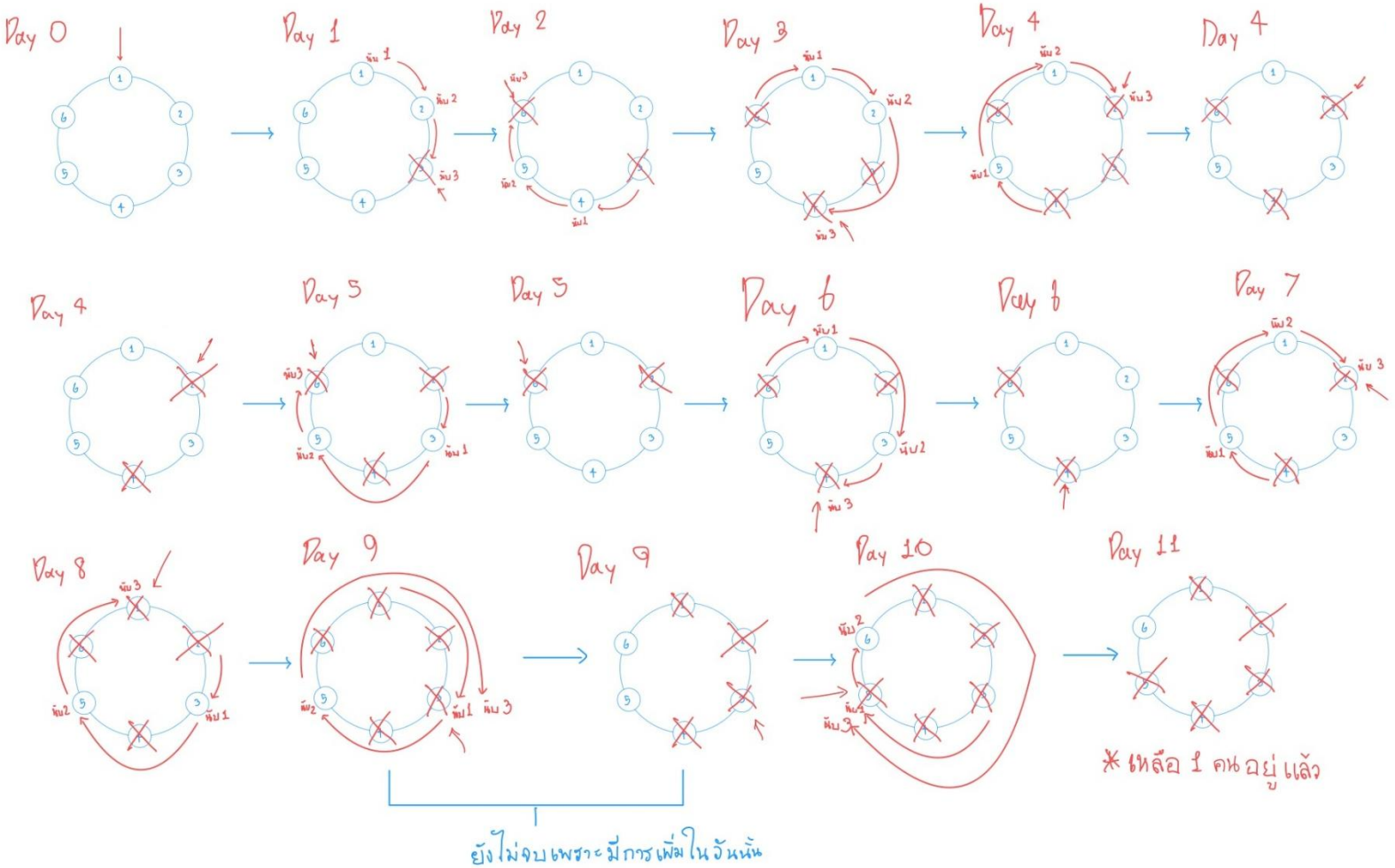
หัวข้อ	เงื่อนไข
ข้อมูลนำเข้า	Standard Input (คีย์บอร์ด)
ข้อมูลส่งออก	Standard Output (จอภาพ)
ระยะเวลาสูงสุดที่ใช้ในการประมวลผล	1 วินาที
หน่วยความจำสูงสุดที่ใช้ในการประมวลผล	64 MB
คะแนนสูงสุดของโจทย์	100 คะแนน

Nam


อธิบายตัวอย่างที่ 1
Day 0
ตำแหน่ง
Day 1
นับ 1
นับ 2
ตำแหน่ง
Day 2
ที่เดิม
นับ 2
ตำแหน่ง
นับ 1
Day 3
ตำแหน่ง
นับ 2
นับ 1
Day 4
ตำแหน่ง
นับ 2
นับ 1
ตอบ 3
อธิบายตัวอย่างที่ 2
Day 0
นับ 1
Day 1
นับ 1
ตำแหน่ง
นับ 2
Day 2
ตำแหน่ง
นับ 2
นับ 1
Day 2
เพิ่มกลับมา
เพราะ 2 ถูกใส่
เป็นคนสุดท้าย
ตำแหน่ง
Day 2
ตำแหน่ง
เพิ่มกลับมา
Day 2 ไม่มีการเพิ่ม (วงเต็มแล้ว)
ตำแหน่ง
Day 3
นับ 2
ตำแหน่ง
นับ 1
Day 4
นับ 1
ตำแหน่ง
นับ 2
Day 4
ตำแหน่ง
Day 5
ตำแหน่ง
นับ 2
นับ 1
Day 6
นับ 1
ตำแหน่ง
นับ 2
Day 7
ตำแหน่ง
นับ 2
นับ 1
Day 8
ตอบ



คำอธิบายตัวอย่างที่ 4



ยังไม่จบ เพราะมีการเพิ่มในวันนั้น

มีคำสั่งใน Day 11 แต่ จับตัวแม่มดสำเร็จไป ก่อนแล้ว จึงไม่มีการ คนมาเพิ่ม